

# DOCUMENTAÇÃO MESTRA

## PORTAL BIOSFERA RÁDIO TV WEB

---

*Versão técnica atualizada - checkpoint e392cba*  
*01/10/2026*

**Projeto:** Portal Biosfera Rádio TV Web

**Responsável técnico:** Marquesano Tecnologia

**Repositório:** ``https://github.com/leomarquesanousa-lab/biosfera-site.git``

**Branch de produção:** ``master``

**Checkpoint de referência:** ``e392cba``

**Stack principal:** Next.js 16.x, React 19, TypeScript, PostgreSQL/Neon

**Ambiente de homologação:** ``https://biosfera.marquesano.com.br``

**Status deste documento:** versão técnica atualizada após implantação do módulo de vídeos e ajustes finais de layout

**Data-base:** 01/10/2026

### 1. Objetivo do sistema

O Portal Biosfera Rádio TV Web foi reconstruído para substituir a solução anterior por uma plataforma própria, moderna, responsiva e administrável, com prioridade para:

- rádio ao vivo;
- TV ao vivo;
- notícias e conteúdo editorial;
- programação;
- vídeos;
- publicidade;
- chat;

- integrações Google;
- SEO;
- administração por perfis;
- manutenção e evolução contínuas.

A arquitetura foi desenhada para que a transmissão ao vivo e o conteúdo editorial não dependam um do outro.

## 2. Estado atual do projeto

### 2.1 Situação geral

A aplicação está funcional em ambiente local e foi enviada ao repositório de produção no commit:

`e392cba`

Esse checkpoint inclui, entre outros itens:

- módulo administrativo de vídeos;
- upload local de vídeos;
- listagem pública de vídeos;
- player público;
- botão de exclusão de vídeo;
- ajustes de autenticação e sessão;
- upload de arquivos;
- melhorias no chat;
- novo favicon;
- ícones de redes sociais;
- ajustes de cabeçalho e área restrita.

### 2.2 Pendência crítica conhecida

**Armazenamento persistente de mídia na Hostinger.**

No ambiente atual, imagens e vídeos locais são gravados no filesystem usado pela aplicação. Em produção, esse caminho não é considerado definitivo porque novos deploys podem trocar a versão ativa da aplicação.

A solução aprovada para este projeto é:

- manter mídia na própria Hostinger;
- usar uma pasta persistente fora da árvore de deploy;
- configurar o caminho por variável de ambiente;
- validar upload e reprodução;
- executar um novo deploy e confirmar que os arquivos permanecem.

**Status:** pendente por indisponibilidade temporária do painel/SSH da Hostinger durante migração/upgrade do plano.

### 2.3 Outras pendências

- exclusão de matérias/notícias no painel ainda precisa ser concluída;
- armazenamento persistente de mídia em produção;
- validação final do timeout de sessão em uso real;
- revisão completa do fluxo de vídeo em produção após correção do storage;
- opcionalmente implementar suporte a HTTP Range para vídeos maiores e seek mais eficiente.

## 3. Arquitetura atual

### 3.1 Frontend e backend

A aplicação utiliza:

- Next.js 16.x;
- App Router;
- React 19;

- TypeScript;
- Server Components;
- Client Components onde há interação;
- Server Actions para operações administrativas;
- Route Handlers para uploads e entrega de mídia.

### 3.2 Banco de dados

Banco atual:

- PostgreSQL;
- hospedado no Neon;
- acesso via pacote `pg`;
- sem Prisma;
- migrations SQL próprias.

### 3.3 Hospedagem

Produção/homologação:

- Hostinger;
- deploy via integração com GitHub;
- branch `master`.

### 3.4 Repositório

Repositório oficial:

``https://github.com/leomarquesanousa-lab/biosfera-site.git``

Branch utilizada:

``master``

## 4. Estrutura principal do projeto

Estrutura lógica atual:

```
app/  
  (public)/  
  admin/  
    (protected)/  
    video-actions.ts  
  api/  
    videos/  
  video-media/  
  icon.png  
  visual-refresh.css
```

```
components/  
  admin/  
  editorial/  
  layout/  
  programming/  
  radio/  
  videos/
```

```
database/  
  migrations/
```

```
lib/  
  auth/  
  db/  
  security/
```

```
server/  
  services/
```

```
scripts/
```

### Arquivos relevantes

- `components/layout/public-header.tsx`
- `components/radio/radio-player.tsx`
- `components/programming/chat.tsx`

- `components/admin/admin-session-guard.tsx`
- `components/videos/public-video-player.tsx`
- `components/videos/video-form.tsx`
- `components/videos/video-upload.tsx`
- `components/videos/video-delete-button.tsx`
- `server/services/videos.ts`
- `server/services/storage.ts`
- `lib/auth/session.ts`

## 5. Variáveis de ambiente

Variáveis já utilizadas ou previstas:

```
DATABASE_URL=
SITE_NAME=Biosfera Rádio TV Web
RADIO_STREAM_URL=https://s1.dvr.ovh:6696/stream
LIVE_CAMERA_EMBED_URL=https://playerv.tvr.ovh/video-premium/video36282/true/false/
PORTAL_TIMEZONE=America/Sao_Paulo
CHAT_ENABLED=true
SITE_URL=https://biosfera.marquesano.com.br
```

### Variável pendente para storage persistente

Quando o acesso à Hostinger retornar, será criada uma variável semelhante a:

```
MEDIA_STORAGE_PATH=/home/USUARIO/biosfera-media
```

O valor real só deve ser definido após consultar o caminho HOME da conta.

## Segurança

Nunca versionar:

- senhas;
- tokens;
- secrets;
- chaves privadas;
- credenciais de banco;
- chaves de APIs externas.

## 6. Banco de dados e migrations

Migrations existentes:

001\_foundation.sql  
002\_editorial\_core.sql  
003\_home\_interactions.sql  
004\_programming\_chat.sql  
005\_admin\_users\_ads.sql  
006\_videos.sql  
007\_videos\_local\_upload.sql  
008\_media\_assets\_video.sql

### 6.1 001 - Foundation

Cria a base inicial da aplicação e estruturas fundamentais.

### 6.2 002 - Editorial core

Estruturas editoriais centrais para matérias/notícias.

### 6.3 003 - Home interactions

Recursos de interação da home.

## 6.4 004 - Programming/chat

Estruturas relacionadas à programação e chat.

## 6.5 005 - Admin/users/ads

Usuários administrativos, funções, permissões e publicidade.

## 6.6 006 - Videos

Cria a tabela `videos`.

## 6.7 007 - Videos local upload

Inclui `LOCAL` como origem de vídeo:

YOUTUBE  
VIMEO  
EXTERNAL  
LOCAL

## 6.8 008 - Media assets video

Permite mídia:

image/webp  
image/jpeg  
image/png  
video/mp4  
video/webm

## Execução

Comando utilizado:

npm.cmd run db:migrate

## 7. Autenticação e controle de acesso

### 7.1 Perfis administrativos

Perfis existentes:

- OWNER;
- ADMIN;
- EDITOR.

### 7.2 Sessão

Arquivo principal:

`lib/auth/session.ts`

Características:

- cookie HTTP only;
- cookie seguro em produção;
- SameSite Lax;
- token randômico;
- hash do token armazenado;
- validação no banco;
- controle de expiração;
- atualização de `last\_seen\_at`.

### 7.3 Timeout por inatividade

Foi introduzido controle de inatividade administrativa com:

```
`components/admin/admin-session-guard.tsx`
```

Objetivo:

- reduzir risco em computador compartilhado;
- encerrar acesso após inatividade;
- exigir novo login.

### 7.4 Pendência de validação

O comportamento final de heartbeat/atividade deve ser validado antes de considerar esse item encerrado definitivamente.

## 8. Painel administrativo

O painel administrativo possui estrutura protegida em:

```
`app/admin/(protected)/`
```

Módulos já trabalhados:

- dashboard;
- notícias/matérias;
- usuários;
- integrações;
- publicidade;

- programação;
- chat;
- vídeos;
- SEO;
- configurações.

### Área restrita

O cabeçalho público possui link:

`/admin`

A versão aprovada mantém apenas um acesso visível no desktop, posicionado após o botão "Ouça ao vivo".

## 9. Notícias / Matérias

O sistema editorial suporta publicação e edição de conteúdo.

Recursos principais:

- título;
- resumo;
- conteúdo;
- imagem;
- categoria;
- autor;
- status;
- destaque;
- publicação;
- SEO.

## Pendência

A listagem de matérias ainda precisa receber uma ação explícita de:

- Excluir;
- confirmação antes da exclusão;
- remoção no banco;
- atualização da listagem.

Esse item não deve ser considerado concluído enquanto não for implementado e testado.

## 10. Módulo de vídeos

### 10.1 Rotas administrativas

```
/admin/videos  
/admin/videos/novo  
/admin/videos/[id]
```

### 10.2 Arquivos

```
app/admin/(protected)/videos/page.tsx  
app/admin/(protected)/videos/novo/page.tsx  
app/admin/(protected)/videos/[id]/page.tsx  
app/admin/video-actions.ts  
components/videos/video-form.tsx  
components/videos/video-upload.tsx  
components/videos/video-delete-button.tsx  
server/services/videos.ts
```

### 10.3 Origens suportadas

- YouTube;
- Vimeo;

- URL externa;
- arquivo local.

## 10.4 Upload

Route Handler:

``app/api/videos/upload/route.ts``

Regras atuais:

- autenticação administrativa;
- validação de origem/host;
- leitura do arquivo;
- limite aproximado de 100 MB;
- aceita MP4 e WebM;
- grava media asset;
- registra auditoria;
- remove arquivo em caso de falha posterior.

## 10.5 Exclusão

A action já suporta:

intent=delete

A listagem possui botão de exclusão com confirmação.

## 10.6 Vídeos públicos

A rota pública ``/videos`` consulta vídeos publicados.

O grid foi ajustado para:

- cards compactos;
- múltiplas colunas em desktop;
- proporção visual 16:9;
- vídeo vertical exibido com `object-fit: contain`.

## 10.7 Player

Componente:

`components/videos/public-video-player.tsx`

Recursos:

- reprodução em overlay;
- tentativa de fullscreen;
- autoplay;
- controles;
- suporte a vídeo local;
- suporte a YouTube/Vimeo;
- ESC para fechar.

## 10.8 Problema atual em produção

Vídeo local não funcionou corretamente em produção.

Causa mais provável:

- arquivo físico salvo em path pertencente à versão do deploy;
- path não persistente entre releases.

Solução aprovada:

- storage persistente na Hostinger;
- caminho externo ao deploy;
- variável de ambiente para o path;
- teste após novo deploy.

## 11. Storage de imagens e vídeos

Arquivo:

`server/services/storage.ts`

### Estado atual

Suporta:

- imagens;
- MP4;
- WebM;
- UUID no nome do arquivo;
- leitura e descarte.

### Limitações atuais

- path local ainda não está apontando para área persistente de produção;
- vídeo é lido integralmente em memória;
- entrega ainda não usa HTTP Range.

### Próxima alteração

Quando Hostinger voltar:

1. acessar SSH;
2. executar:

```
``bash

pwd

echo $HOME

``
```
3. criar pasta persistente;
4. definir `MEDIA\_STORAGE\_PATH`;
5. atualizar storage;
6. testar upload;
7. testar reprodução;
8. redeploy;
9. confirmar persistência.

## 12. Rádio ao vivo

Stream:

``https://s1.dvr.ovh:6696/stream``

Objetivos implementados:

- player persistente;
- controles;
- reprodução durante navegação;
- botão "Ouça ao vivo".

O player deve continuar desacoplado do sistema editorial.

### 13. TV / câmera ao vivo

Embed atual:

```
`https://playerv.tvr.ovh/video-premium/video36282/true/false/`
```

Variável:

LIVE\_CAMERA\_EMBED\_URL

A integração deve permanecer configurável e não hardcoded em múltiplos componentes.

### 14. Programação

O portal possui área de programação com conceito:

- agora;
- depois;
- grade.

Quando não existe programação cadastrada, o frontend deve apresentar estado vazio de forma limpa.

## 15. Chat

Arquivos principais:

`components/programming/chat.tsx`

O chat teve ajustes visuais e de fluxo.

Características:

- ativação por configuração;
- interação pública;
- área administrativa relacionada;
- ações de foto/vídeo só aparecem após entrada no chat.

### Observação

Atualmente os botões de mídia não representam um sistema completo de anexos persistentes. Upload real de mídia do chat deve ser tratado como evolução futura.

## 16. Publicidade

O sistema possui módulo de publicidade/anúncios.

Itens previstos ou existentes:

- banners;
- campanha;

- posição;
- data;
- link;
- status;
- integração administrativa.

As artes e banners devem ser responsivos e não podem quebrar o layout em diferentes proporções.

## 17. Google Analytics

GA4 foi configurado e validado.

Status:

- integração ativa;
- teste real concluído;
- exibido como configurado no admin.

## 18. Google Search Console

A propriedade foi adicionada e o acesso de leitura foi corrigido.

Status:

- leitura ativa;
- integração funcional.

## 19. Google AdSense

Existe integração administrativa para AdSense.

Foi observado warning no navegador:

AdSense head tag doesn't support data-nscript attribute

Esse warning está relacionado ao script injetado pelo Next.js e não ao favicon.

O comportamento deve ser monitorado em produção, mas o warning isolado não significa necessariamente falha do site.

## 20. SEO

O portal recebeu:

- metadata global;
- metadata dinâmica;
- sitemap;
- robots;
- canonical;
- JSON-LD;
- configuração de indexação.

A busca interna não deve ser indexada.

Rotas administrativas não devem ser indexadas.

## 21. Favicon

O favicon antigo:

``app/favicon.ico``

foi removido.

O favicon atual:

``app/icon.png``

é utilizado pelo App Router como ícone da aplicação.

Se o navegador exibir o ícone anterior, verificar cache antes de alterar código.

Teste direto:

`http://localhost:3000/icon.png`

## 22. Redes sociais

Links configurados:

- Facebook: `https://www.facebook.com/biortv.official/`
- Instagram: `https://www.instagram.com/ritvit26`
- X: `https://x.com/biortv72344`
- YouTube: `https://www.youtube.com/@MultbioRTV`

### Layout aprovado

No desktop:

- ícones flutuantes na lateral direita;
- fora do container principal;
- não ocupam espaço da navegação.

No mobile:

- a barra lateral é ocultada.

A implementação aprovada está diretamente em:

`components/layout/public-header.tsx`

Não alterar essa estrutura sem necessidade.

## 23. Cabeçalho

Arquivo:

`components/layout/public-header.tsx`

Itens atuais:

- logo;
- Início;
- Notícias;
- Rádio;
- Programação;
- Vídeos;
- Contato;
- Busca;
- Ouça ao vivo;
- Área restrita;
- redes sociais laterais.

### Regra importante

Não duplicar o link "Área restrita".

A versão aprovada possui apenas o acesso desktop após "Ouça ao vivo".

## 24. CSS e layout

Arquivo principal:

`app/visual-refresh.css`

### Incidente registrado

Durante uma alteração pontual, o arquivo foi substituído parcialmente e caiu de centenas de linhas para poucas dezenas, causando quebra geral do layout.

Foi restaurado a partir do Git.

### Regra operacional obrigatória

**Nunca aplicar trecho parcial diretamente em arquivo existente sem entregar o arquivo completo.**

Para este projeto:

- alterações de arquivo devem ser entregues como arquivo inteiro;
- informar sempre o caminho exato;
- se o conteúdo atual não estiver disponível, solicitar o arquivo antes de modificar;
- não mandar blocos parciais para colar no início, meio ou fim.

Essa regra deve ser respeitada em todas as futuras alterações.

## 25. Uploads editoriais

Arquivos envolvidos:

components/editorial/image-upload.tsx  
components/editorial/file-upload.tsx  
server/services/storage.ts

Objetivo visual:

- área de drag-and-drop;
- upload por clique;
- feedback;
- preview;
- experiência consistente.

A implementação deve continuar evoluindo de forma reutilizável.

## 26. Segurança

Diretrizes:

- cookies HTTP only;
- secure em produção;
- segredos via env;
- validação server-side;
- autenticação administrativa;
- autorização por função;
- auditoria de alterações sensíveis;
- validação de arquivo/mime;
- evitar armazenamento de credenciais em código;
- evitar logs com secrets.

## 27. Deploy

### Fluxo atual

1. desenvolvimento local;
2. build;
3. commit;
4. push para `master`;
5. Hostinger puxa o repositório;
6. deploy;
7. validação.

## Último push documentado

f2de68a..e392cba  
master -> master

## Comandos

```
git status  
npm.cmd run build  
git add -A  
git commit -m "..."  
git push origin master
```

## 28. Checklist de deploy

Antes do push:

- ☐ `git status`
- ☐ build sem erro
- ☐ home
- ☐ header
- ☐ favicon
- ☐ rádio
- ☐ vídeos
- ☐ notícias
- ☐ login/admin
- ☐ área restrita
- ☐ programação
- ☐ chat
- ☐ banners
- ☐ mobile

Após deploy:

- ☐ abrir produção

- ☐ validar home
- ☐ validar rádio
- ☐ validar vídeo
- ☐ validar admin
- ☐ validar upload
- ☐ validar favicon
- ☐ validar redes sociais
- ☐ conferir logs

## 29. Problemas conhecidos

### 29.1 Storage de mídia

**Status:** crítico/pendente.

Arquivos locais podem não sobreviver a novo deploy.

### 29.2 Vídeo em produção

**Status:** pendente por storage.

### 29.3 Exclusão de matéria

**Status:** pendente.

### 29.4 Timeout administrativo

**Status:** implementado, requer validação final.

### 29.5 HTTP Range

**Status:** não implementado.

Pode ser útil para:

- seek;
- vídeos maiores;
- redução de leitura desnecessária.

## 30. Checkpoint atual

### Commit

`e392cba`

### Considerado aprovado

- layout público;
- header;
- redes sociais laterais;
- favicon novo no código;
- módulo administrativo de vídeos;
- player público;
- upload local em desenvolvimento;
- exclusão de vídeo;
- migrations 006, 007 e 008;
- integrações Google anteriores;
- SEO anterior;
- usuários/admin anteriores.

### Pendente antes da entrega definitiva

1. storage persistente Hostinger;
2. reteste de vídeo em produção;

3. exclusão de matéria;
4. validação final de sessão;
5. checklist completo de produção.

## 31. Procedimento assim que a Hostinger voltar

### Etapa 1 - descobrir o ambiente

No SSH:

```
pwd  
echo $HOME
```

### Etapa 2 - criar pasta persistente

Exemplo conceitual:

```
/home/USUARIO/biosfera-media
```

O path real deve ser confirmado antes.

### Etapa 3 - variável de ambiente

```
MEDIA_STORAGE_PATH=
```

### Etapa 4 - ajustar código

Revisar arquivos completos:

```
server/services/storage.ts  
app/api/videos/upload/route.ts  
app/video-media/[filename]/route.ts
```

## Etapa 5 - teste obrigatório

1. subir vídeo;
2. reproduzir;
3. fazer novo deploy;
4. reproduzir o mesmo vídeo novamente.

Somente após esse teste o storage deve ser considerado concluído.

## 32. Manutenção

Recomendações:

- commits pequenos;
- checkpoint estável antes de alterações grandes;
- não misturar refactor com correção urgente;
- build antes de push;
- backup do banco;
- manter migrations versionadas;
- documentar env vars;
- registrar pendências imediatamente;
- não alterar layout global para resolver problema local.

## 33. Regra de desenvolvimento para continuidade

Para qualquer nova alteração:

1. identificar arquivo exato;
2. ler versão atual;
3. alterar o mínimo necessário;
4. entregar arquivo completo;
5. testar local;
6. build;
7. commit;
8. deploy;
9. validar produção.

Nunca assumir que um arquivo parcial representa o arquivo completo.

## 34. Próximas melhorias possíveis

Após a entrega:

- storage persistente;
- Range requests;
- otimização de vídeo;
- thumbnails automáticas;
- anexos reais no chat;
- histórico editorial;
- exclusão de matérias;
- observabilidade;
- logs de mídia;
- métricas de vídeo;
- CDN futura, se o volume justificar.

## 35. Fonte de verdade técnica

Ordem recomendada:

1. código no commit estável mais recente;
2. migrations;
3. este documento;
4. configurações de produção;
5. decisões registradas posteriormente.

Quando houver divergência entre documentação e código, verificar primeiro o commit implantado e atualizar este documento.

## 36. Observação final

Este documento substitui a documentação de planejamento inicial como referência técnica do estado real da aplicação em 01/10/2026.

A documentação inicial continua útil como histórico de escopo e decisões, mas esta versão deve ser usada para manutenção, continuidade e handoff técnico.